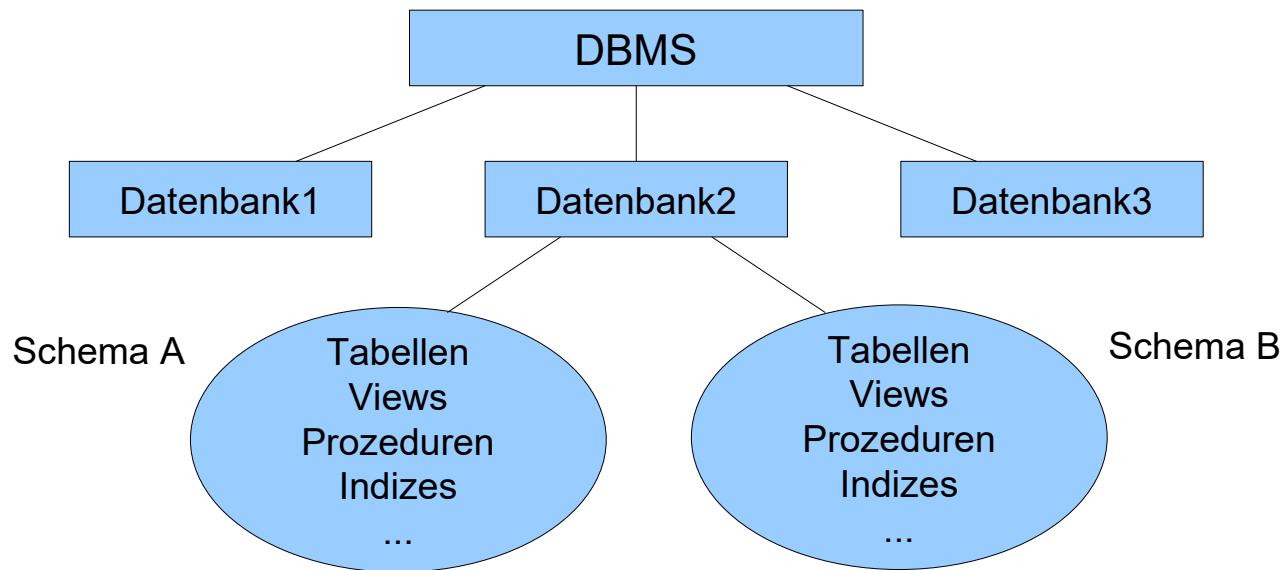




MariaDB – Einfache Abfragen

Stephan Karrer

Schemata fassen Datenbankobjekte zu logischen Gruppen zusammen



- Entspricht einem Verzeichnis im Dateisystem, allerdings in der Regel ohne Schachtelung (so auch MariaDB).
- Bei MariaDB ist Datenbank mit Schema identisch.
- Die Hersteller setzen allerdings Schemata durchaus unterschiedlich um.
- Die jeweilige Umsetzung hat nur grob etwas mit dem Schema-Begriff des Datenbankentwurfs zu tun.

Schema-Umsetzung bei MariaDB

- Schema und Datenbank sind identisch
- Ein Benutzer kann mehrere Datenbanken benutzen, sofern er die Berechtigung dazu hat.
- Adressierung eines Datenbank-gebundenen Objekts, z.B. einer Tabelle erfolgt via Datenbank-Name.Tabellen-Name, z.B:

```
SELECT      *      FROM  HR.EMPLOYEES
```

- Wir können die Qualifizierung aber einsparen, z.B:

```
USE hr;
```

Abfragen mit SQL: SELECT-Anweisung

```
SELECT [ALL|DISTINCT] Auswahlliste
      FROM Quelle
      [WHERE Where-Klausel]
      [ORDER BY (Sortierungsattribut) [ASC|DESC]]
```

```
SELECT * FROM employees;
SELECT last_name, job_id, salary, department_id
      FROM employees;
SELECT first_name AS `Vorname 1`, last_name "Nach""name"
      FROM employees;
```

- Generell gilt: SQL ist nicht Case-Sensitiv. Schlüsselworte (wie SELECT, FROM, ...) und nicht-maskierte (unquoted) Namen können beliebig groß/klein geschrieben werden.
- Wollen wir Namen case-sensitiv bzw. in den Namen nicht-konforme Zeichen verwenden, können wir diese maskieren. (" ist ANSI, ` ist MariaDB)
- Generell sind Spalten-Aliase für die angelieferten Spalten möglich.

SELECT-Anweisung mit Sortierung

```
SELECT [ALL|DISTINCT] Auswahlliste
      FROM Quelle
      [WHERE Where-Klausel]
      [ORDER BY (Sortierungsattribut) [ASC|DESC]]
```

```
SELECT last_name, job_id, department_id FROM employees
      ORDER BY department_id, last_name DESC;
```

```
SELECT last_name, job_id FROM employees
      ORDER BY department_id, last_name DESC;
```

```
SELECT DISTINCT job_id FROM employees;
```

- Es sind durchaus mehrere Sortier-Kriterien mit expliziter Angabe absteigend/aufsteigend möglich.
- Die Reihenfolge der Ausgabespalten muss nicht der Reihenfolge der Sortierkriterien entsprechen bzw. diese überhaupt enthalten.
- Vorsicht! Sortierung kostet Performance, also nicht unnötig sortieren
- DISTINCT bedingt in der Regel intern Sortierung!

Abfragen mit SQL: SELECT-Anweisung mit Ausdrücken

```
SELECT last_name, salary, 12*(salary+100) AS new_annual_sal  
FROM employees;
```

```
SELECT last_name "employees in department 50"  
FROM employees WHERE department_id = 50;
```

```
SELECT DISTINCT job_id FROM employees WHERE salary <= 5000;
```

```
-- reine Berechnung  
SELECT (2+8)/5 "Ergebnis";
```

- Überall dort, wo ein Wert erwartet wird, darf auch ein Ausdruck stehen.
- Natürlich hängt die Funktionalität vom jeweiligen Datentyp ab.
- Runde Klammern regeln wie üblich Ausführungsreihenfolge.
- FROM-Klausel ist bei reinen Berechnungen obsolet!

Abfragen mit VALUES-Klausel

```
SELECT * FROM (VALUES (1, 'one'), (2, 'two'), (3, 'three')) t;  
  
SELECT t.num+10, t.letter FROM  
    (VALUES (1, 'one'), (2, 'two'), (3, 'three')) AS t (num,letter);
```

- Mit Hilfe der VALUES-Klausel kann schnell eine kleine Tabelle im Speicher erstellt werden.
- Jedes Tupel definiert eine Zeile, daher müssen alle Tupel die gleiche Struktur haben!
- Sollen die einzelnen Spalten gezielt angesprochen werden, empfehlen sich Aliase zusätzlich zum Table-Alias.
- Ideal zum Ausprobieren am kleinen Beispiel.

ANSI SQL Basis-Datentypen

CHARACTER CHARACTER VARYING (or VARCHAR) CHARACTER LARGE OBJECT NCHAR NCHAR VARYING	NUMERIC DECIMAL SMALLINT INTEGER BIGINT FLOAT REAL DOUBLE PRECISION
BINARY BINARY VARYING BINARY LARGE OBJECT	DATE TIME TIMESTAMP INTERVAL
BOOLEAN	

Kaum ein Hersteller setzt das 1:1 um !

(Bei den komplexeren Typen: Object, XML, ... ist die Umsetzung noch unsicherer)

Siehe auch: https://en.wikibooks.org/wiki/SQL_Dialects_Reference

Zeichenketten als Datentyp

Name	Beschreibung
character varying(n), varchar(n)	Variable Länge mit Max. n (n < 65536)
character(n), char(n), bpchar(n)	Fixe Länge n, default 1
text	Variable Länge ohne direktes Limit (< 65536)

```
SELECT 'Max Muster'; -- z.B. für VARCHAR(15) oder TEXT
SELECT 'otto';       -- z.B. für CHAR(4)
SELECT 'Mother''s Day'; -- Escape
```

- TEXT ist der native MariaDB-Typ, die anderen existieren zwecks ANSI-Konformität.
- VARCHAR ohne Längenangabe entspricht TEXT.
- CHAR hat fixe Länge, wird erforderlichenfalls mit NULL-Bytes aufgefüllt.
- Es gibt weitere: <https://mariadb.com/docs/server/reference/data-types/string-data-types>
- Länge wird in Zeichen gemessen, abhängig vom Zeichensatz (Datenbank-Parameter) können deutlich mehr Bytes benötigt werden.

Ganzzahlen als Datentyp

Name	Speichergröße	Bereich
tinyint (int1)	1 Byte	-128 to +127
smallint (int2)	2 Bytes	-32768 to +32767
integer (int4)	4 Bytes	-2147483648 to +2147483647
bigint (int8)	8 Bytes	-9223372036854775808 to +9223372036854775807

```
SELECT -1 AS erg;      -- liefert -1
SELECT 2*2 AS erg;     -- liefert 4
SELECT 2+3 AS erg;     -- liefert 5
SELECT 7-3 AS erg;     -- liefert 4
SELECT 7/3 AS erg;     -- Division: liefert 2.3333
SELECT 7%3 AS erg;     -- Modulo-Operator: liefert 1
```

- Es stehen die üblichen arithmetischen Operatoren zur Verfügung.

Spezielles für Ganzzahlen

```
SELECT 91 & 15 AS erg;      -- Bitwise AND: liefert 11
SELECT 32 | 3 AS erg;       -- Bitwise OR: liefert 35
SELECT 17 ^ 5 AS erg;       -- Bitwise exclusive OR: liefert 20
SELECT 3 & ~1 AS erg;       -- Bitwise NOT: liefert 2
SELECT 1 << 4 AS erg;       -- Bitwise Shift Left: liefert 16
SELECT 8 >> 2 AS erg;       -- Bitwise Shift Right: liefert 2
```

- Man kann auf die Ganzzahlen auch BIT-Operationen anwenden.
Sofern dann noch der Durchblick vorhanden !

Gleitpunktzahlen als Datentyp

Name	Speichergröße	Bereich
numeric (decimal)	variabel	Precision max. 65 (Default 10) Scale max. 38 (Default 0)
float	4 Bytes	ca. 1E-37 bis 1E+37 mit einer Präzision von 6 Dezimalstellen
double precision (real , double)	8 Bytes	ca. 1E-307 bis 1E+308 mit einer Präzision von 15 Dezimalstellen

```
SELECT 3.5 + .001 AS erg;           -- liefert 3.501
SELECT 5e2 - 1.1e-3 AS erg;        -- liefert 499.9989
SELECT 2 * 3.5 AS erg;             -- liefert 7.0
SELECT 7.0 / 2.0 AS erg;           -- liefert 3.5
SELECT 7.0 % 3 AS erg;              -- Modulo-Operator: liefert 1.0
SELECT CAST(0.1 AS DOUBLE) + CAST(0.2 AS DOUBLE) ;
-- liefert 0.300000000000000004
```

- Wissenschaftliche und numerische Schreibweise können beliebig gemischt werden.
- Die Gesamtzahl der Stellen wird Precision genannt, die Anzahl Nachkommastellen Scale.
- Das IEEE-Format (float, double precision) ist intern ein Binärformat und nicht für kaufmännische Berechnungen geeignet !!

ANSI Datentypen: Datum und verschiedene Zeitstempel

<u>ANSI</u>	<u>Beschreibung</u>
DATE	nur Datum
TIME	nur Zeit
DATETIME	Datum und Zeit ohne Zeitzone
TIMESTAMP	Datum und Zeit mit Umrechnung in UTC-Zeitzone (intern Sekunden seit 1970-01-01 00:00:00' UTC)
YEAR	nur Jahr

- Es erfolgt keine Speicherung von Zeitzonen.
- Es gibt keine Intervall-Typen.

Datum und Zeit: Eingabe-Beispiele

```
SELECT  DATE  '2003-06-17';
```

```
SELECT  TIME  '13:05:06';
```

```
SELECT  TIME  '13:05';
```

```
SELECT  TIME  '13:05:06.45';
```

```
SELECT  TIMESTAMP  '2003-06-17T13:45:30';
```

```
SELECT  TIMESTAMP  '2003-06-17T13:45:30.67';
```

- Die ISO 8601 – Formate sind am portabelsten.

Wahrheitswerte

Name	Speicher	Bereich
boolean	1 Byte	TRUE, FALSE

```
SELECT TRUE OR FALSE;
SELECT TRUE AND FALSE;
SELECT NOT TRUE;
SELECT * FROM employees WHERE TRUE;
SELECT (1=1) = TRUE;
SELECT 1=1 IS TRUE;
SELECT 1=1 IS NOT FALSE;
```

- Ist aber nur Synonym für Tinyint:
0 ist FALSE, alles andere ist TRUE

Vergleichsoperatoren für alle Datentypen mit Ordnung

=	gleich
<>, !=	ungleich
>	größer
>=	größer oder gleich
<	kleiner
<=	kleiner oder gleich

```
SELECT * FROM employees  
WHERE salary = 2500;
```

```
SELECT * FROM employees  
WHERE salary != 2500;
```

```
SELECT * FROM employees  
WHERE salary > 2500;
```


Logik-Operatoren

- Logische Verknüpfungsoperatoren können auf logische Ausdrücke angewendet werden.

Operator	Kommentar
AND, OR, NOT	Basisoperatoren

```
SELECT * FROM employees
WHERE NOT (job_id IS NULL)
ORDER BY employee_id;
```

```
SELECT * FROM employees
WHERE job_id = 'PU_CLERK' AND department_id = 30;
```

Spezielle Vergleichsoperatoren

Operator	Kommentar
BETWEEN	Prüft, ob der Operand im Intervall liegt
IN	Prüft, ob der Operand in der Aufzählung enthalten ist
LIKE	Prüft, ob der Operand einem Muster gleicht

```
SELECT * FROM employees
      WHERE salary BETWEEN 5000 AND 10000;
```

```
SELECT * FROM employees
      WHERE job_id IN ('SA_MAN', 'SA_REP');
```

```
SELECT * FROM employees
      WHERE (first_name, last_name, email) IN
            (('Guy', 'Himuro', 'GHIMURO'),
             ('Karen', 'Colmenares', 'KCOLMENA'));
```

- Tupelvergleiche sind verfügbar.

LIKE-Operator für Vergleiche

```
x [NOT] LIKE y [ESCAPE 'z']
```

Zur Bildung von Mustern können verwendet werden:

- % beliebig viele Zeichen (auch keines)
- _ genau ein Zeichen

```
SELECT salary
  FROM employees
 WHERE last_name LIKE 'R%';
```

```
SELECT last_name
  FROM employees
 WHERE last_name LIKE '%A\_B%' ;  -- \ ist default
```

- LIKE ist rudimentär, deshalb bieten viele DBMS wie auch MariaDB Unterstützung von regulären Ausdrücken an.

Nullwerte (Null Values)

- Nullwerte stehen für nicht verfügbare bzw. unbekannte Werte und können in Tabellen als Werte von Spalten vorkommen
- Werte können explizit auf NULL gesetzt bzw. daraufhin überprüft werden
- Ist ein Operand in arithmetischen Ausdrücken ein Nullwert, so ergibt die Auswertung stets NULL.
- Vergleiche mit Nullwerten liefern stets NULL (außer die speziellen Tests auf Nullwerte)
- Bei der Auswertung logischer Ausdrücke wird durch Nullwerte die Prädikatenlogik erweitert.

Vorsicht bei Null-Values!

```
SELECT 1 WHERE 1 = null;  
    -- trifft nichts, ist aber kein Fehler!  
    -- NULL in einem Ausdruck resultiert in NULL als Ergebnis
```

Aber folgende Ausnahmen:

```
SELECT 1 Spalte1 WHERE NULL OR TRUE;  
-- NULL OR TRUE liefert TRUE!
```

```
SELECT Last_name FROM employees  
    WHERE NOT (1 = 2 AND NULL);  
-- NULL AND FALSE liefert FALSE!
```

- Bei der Auswertung logischer Ausdrücke wird durch Nullwerte die Prädikatenlogik erweitert.
 - Dies kann in komplexeren Szenarien leider ein „Kopfschmerz-Thema“ werden!

Prüfung auf NULL-Wert

- Die blau eingefärbten Varianten sind PostgreSQL-spezifisch.

Operator	Kommentar
IS NULL	Prüft, ob der Operand ein NULL-Wert ist
IS NOT NULL	Prüft, ob der Operand kein NULL-Wert ist

```
SELECT last_name
  FROM employees
 WHERE commission_pct IS NULL
 ORDER BY last_name;
```

NULL-Werte berücksichtigen oder nicht

```
SELECT employee_id, last_name, commission_pct FROM employees  
WHERE commission_pct != 0.35;
```

-- oder

```
SELECT employee_id, last_name, commission_pct FROM employees  
WHERE commission_pct != 0.35 OR commission_pct IS null;
```

- Was ist semantisch korrekt?

Typ-Konvertierung (CAST)

ANSI: *CAST (expression AS type)*

MariaDB: *CONVERT (expression, type)*

```
SELECT CAST('12.345' AS DECIMAL); -- liefert 12
```

```
SELECT CONVERT('12.345', DECIMAL); -- liefert 12
```

- Es muss als Typ der eigentliche Typ und kein Alias (z.B. Numeric) verwendet werden!
- Für die Konversion von/zu Zeichenketten bei Zahlen und vor allem Datums- und Zeit-Werten existieren spezielle Varianten, die spezielle Formatierungen berücksichtigen.

SQL-Funktionen: Numerik (Auszug)

Funktion	Beschreibung
ABS(zahl)	Absolutbetrag
CEILING(zahl)	Nächstgrößere Ganzzahl
FLOOR(zahl)	Nächstkleinere Ganzzahl
ROUND(zahl [, n])	Runden auf n Stellen
TRUNC(zahl [, n])	Abschneiden auf n Stellen
DIV(zahl1, zahl2)	Ganzzahl-Division
MOD(zahl1, zahl2)	Rest-Operation (Modulo)
SQRT(zahl)	Quadratwurzel
POWER(zahl, n)	n-te Potenz von zahl
...	und viele weitere (siehe Doku)

SQL-Funktionen: Zeichenketten (Auszug)

Funktion	Beschreibung
LOWER(text)	Konvertierung zu Kleinbuchstaben
UPPER(text)	Konvertierung zu Großbuchstaben
LENGTH(text)	Länge der Zeichenkette in Bytes
SUBSTR (text, n1 [, n2])	Teilzeichenkette von Position n1 bis Position n2
LEFT(text, n)	Linke Teilzeichenkette der Länge n
RIGHT(text, n)	Rechte Teilzeichenkette der Länge n
CONCAT(text1, text2, ...)	Konkatenation
LPAD / RPAD(text, n [, pads]	Links bzw. rechts auffüllen auf Länge n mit Zeichenkette pads (default Leerzeichen)
LTRIM / RTRIM(text [, chars])	Links bzw. rechts abschneiden der längsten Zeichenkette, die nur aus Zeichen von chars besteht.
...	und viele weitere (siehe Doku)

SQL-Funktionen: Zeitstempel (Auszug)

Funktion	Beschreibung
CURRENT_DATE CURRENT_TIME [(precision)] CURRENT_TIMESTAMP[(precision)]	Aktuelles Datum bzw. Zeit, optionale precision reduziert die Sekundenbruchteile
LOCALTIME [(precision)] LOCALTIMESTAMP [(precision]	Aktuelle Zeit bzgl Session-Zeitzone
EXTRACT(field FROM source)	Extrahiert Datums bzw. Zeitanteil, Vielzahl von field-Parametern möglich
DATE_TRUNC (field, timestamp)	Reduziert auf die durch field angegebene Genauigkeit
DATE_ADD(date,INTERVAL expr unit)	Addiert das Interval
DATE_SUB(date,INTERVAL expr unit)	Subtrahiert das Interval
...	und einige weitere (siehe Doku)

Funktionen für NULL-Behandlung

Funktion	Beschreibung
COALESCE(value [, ...])	Liefert den ersten Wert, der nicht NULL ist, sofern alle Werte NULL sind ist das Ergebnis NULL
IFNULL(expr1,expr2), NVL(expr1,expr2)	Liefert expr1 falls nicht Null, ansonsten expr2
NULLIF(value1, value2)	Liefert NULL, falls value1 = value2, ansonsten value1

```
SELECT last_name,  
       COALESCE(commission_pct||',', 'Not Applicable') "commission"  
FROM employees ORDER BY last_name;
```

```
SELECT last_name,  
       salary * (1 + COALESCE(commission_pct, 0)) "totalsal"  
FROM employees ORDER BY last_name;
```

- Die Werte müssen einen gemeinsamen Typ haben !

Größter und Kleinster Wert

Funktion	Beschreibung
GREATEST(value [, ...])	Liefert den größten Wert aus einer Menge
LEAST(value [, ...])	Liefert den kleinsten Wert aus einer Menge

```
SELECT GREATEST(2*5, 17-8, NULL, 3*4);  
SELECT LEAST(2*5, 17-8, 3*4);
```

- Die Werte müssen einen gemeinsamen Typ haben !
- NULL-Werte werden nicht ignoriert.

CASE – Ausdruck

```
SELECT  last_name,  
        CASE salary  
          WHEN 2000 THEN 'Low'  
          WHEN 5000 THEN 'High'  
          ELSE 'Medium'  
        END AS sal  
FROM employees;
```

- Der ELSE-Zweig ist optional, nicht getroffene Werte werden nicht ersetzt.
- Auch hier gilt: Die einzelnen Zweige müssen einen gemeinsamen Typ liefern !
- Wurde in der Vergangenheit oft für das Aufbereiten der Ausgabe im Sinne von Reporting benutzt.

Allgemeiner CASE - Ausdruck (Searched CASE)

```
SELECT  last_name,  
        CASE  WHEN salary < 2000 THEN 'Low'  
              WHEN salary > 5000 THEN 'High'  
              ELSE 'Medium' END AS sal  
FROM employees;
```

- Der ELSE-Zweig ist optional, nicht getroffene Werte werden nicht ersetzt.
- Auch hier gilt: Die einzelnen Zweige müssen einen gemeinsamen Typ liefern !
- Ist natürlich viel flexibler, da beliebige Bedingungen formulierbar sind.

TOP-N mit MariaDB

```
SELECT * FROM employees ORDER BY job_id LIMIT 7;

SELECT * FROM employees ORDER BY job_id LIMIT 2 OFFSET 10;

-- ab PostgreSQL 13 (SQL:2008)
SELECT * FROM employees ORDER BY job_id FETCH FIRST 10
                                     ROWS ONLY;

SELECT * FROM employees ORDER BY job_id FETCH FIRST 10
                                     ROWS WITH TIES;
```

- Die LIMIT-Klausel schneidet einfach ab. Das ist nicht so ganz ok, falls es mehrere gleiche Ergebnisse auf der letzten Position gibt.
- Deshalb besser die Variante mit der FETCH-Klausel verwenden.